

Rapport de Projet IDM

Spécification et Validations de Circuits Électroniques

Présenté par :

Fatma Zohra CHAKROUN, Nejla CHELLAL, Abdallah BENALI

Groupe N°4

Année Universitaire 2025-2026

Table des matières

1	Introduction	3
2	Métamodélisation (Ecore)	4
2.1	Le Catalogue	4
2.2	La Netlist	4
2.3	Le Layout	5
3	Édition graphique et Visualisation (Sirius)	6
3.1	Visualisation du Catalogue	6
3.2	Éditeur de Netlist	6
3.3	Visualisation du Layout	7
4	Chaîne de Transformations	8
4.1	Transformation Modèle-vers-Modèle (ATL)	8
4.2	Transformations Modèle-vers-Texte (Acceleo)	9
4.2.1	Génération de la nomenclature (BOM)	9
4.2.2	Génération du rendu vectoriel (SVG)	9
5	Validation et Cohérence	11
5.1	DSL de Contraintes (Xtext)	11
5.2	Vérifications Java : Alternative à l'OCL	11
5.2.1	Contraintes Métier des Modèles	11
5.2.2	Vérificateur de Cohérence XMI (Layout vs Netlist)	12
6	Regard critique	12
6.1	Ergonomie et Passage à l'échelle	12
6.2	Manquements et Améliorations possibles	12
7	Conclusion et Bilan	12
7.1	Bilan technique et difficultés	12
8	Description détaillée du rendu	13
8.1	Dossier : metamodels	13
8.2	Dossier : sirius	13
8.3	Dossier : xtext	13
8.4	Dossier : validation	13
8.5	Dossiers : acceleo et atl	14
8.6	Dossier : vérification de cohérence	14

Table des figures

1	Diagramme de classe Ecore du Catalogue	4
2	Diagramme de classe Ecore de la Netlist	5
3	Diagramme de classe Ecore du Layout	5
4	Éditeur graphique du Catalogue sous Sirius	6
5	Représentation graphique d'une Netlist (connectivité)	7
6	Vue de placement spatial (Layout)	8
7	Modèle Netlist d'origine et modèle Layout résultant	9
8	Livrables générés : Fichier BOM	9
9	Livrables générés : rendu SVG du circuit	10
10	Éditeur Xtext et grammaire des contraintes du catalogue	11

I Introduction

Ce projet s'inscrit dans le cadre de la conception de systèmes embarqués critiques. L'objectif est de fournir une chaîne d'outils basée sur l'Ingénierie Dirigée par les Modèles (IDM) pour sécuriser la conception de circuits électroniques, de leur définition au catalogue jusqu'à leur placement physique.

2 Métamodélisation (Ecore)

La première étape de notre projet a consisté à définir la structure de données fondamentale. Nous avons fait le choix de concevoir trois métamodèles distincts mais complémentaires. Cette modularité nous a permis de gérer indépendamment la bibliothèque de composants, la logique de connexion et la réalité physique du circuit.

2.1 Le Catalogue

Nous avons conçu le métamodèle Catalogue comme une base de données de référence. Il définit de manière abstraite les types de composants disponibles (résistances, microcontrôleurs, etc.) sans notion d'instance.

- **Éléments clés :** Nous y avons intégré la définition des ports, les contraintes techniques (puissance, tension) et les empreintes physiques.

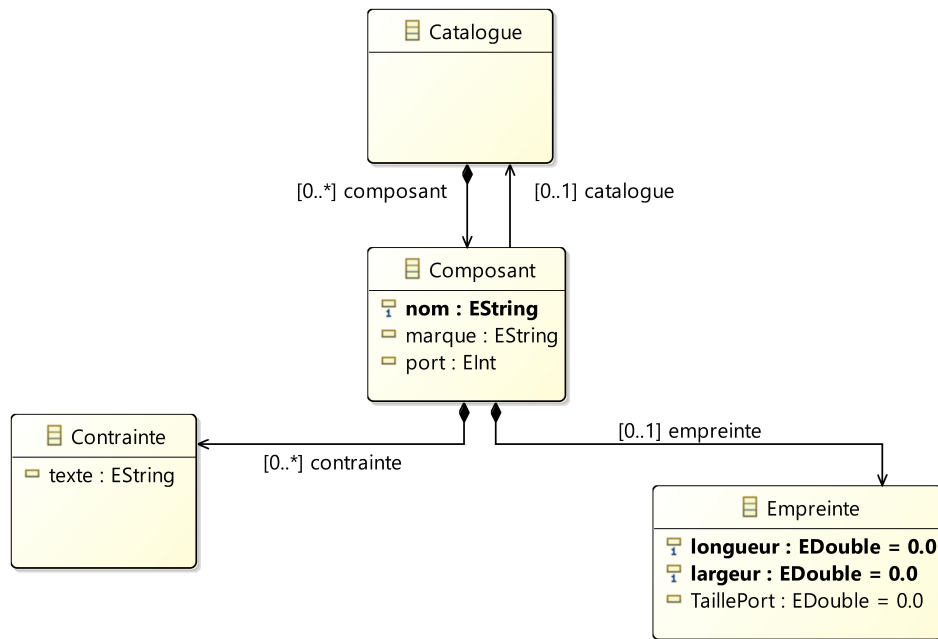


FIGURE 1 – Diagramme de classe Ecore du Catalogue

2.2 La Netlist

Le métamodèle Netlist représente l'aspect logique d'un circuit spécifique. Nous avons pu y implémenter la notion d'instanciation : une Netlist fait référence aux composants du Catalogue pour décrire un assemblage précis.

- **Éléments clés :** Ce modèle se concentre sur les "Nets" (réseaux de connexion) qui relient les ports des différentes instances de composants.
- **Regard critique :** La principale difficulté a été de gérer la navigation entre le modèle Netlist et le Catalogue. Nous avons opté pour des références croisées afin d'assurer que chaque port utilisé dans une connexion existe réellement dans la définition du composant.

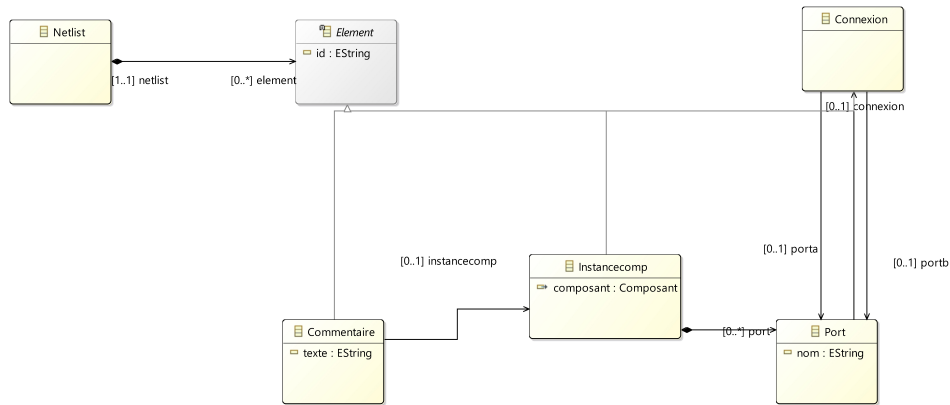


FIGURE 2 – Diagramme de classe Ecore de la Netlist

2.3 Le Layout

Enfin, nous avons développé le métamodèle Layout pour représenter la réalisation physique sur une carte (board). C'est ici que la géométrie entre en jeu.

- **Éléments clés** : Nous avons intégré des attributs de positionnement (coordonnées X, Y) et d'orientation pour chaque composant. Les connexions logiques de la Netlist deviennent ici des "Pistes" physiques.
- **Analyse** : Ce modèle est le plus proche du produit final. Sa conception a été guidée par la nécessité de pouvoir générer, par la suite, un rendu visuel précis (SVG).

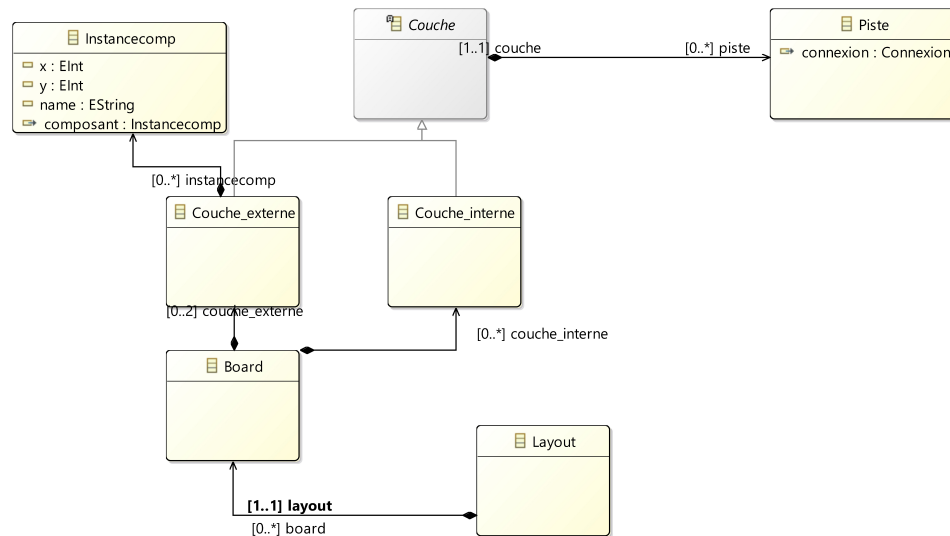


FIGURE 3 – Diagramme de classe Ecore du Layout

3 Édition graphique et Visualisation (Sirius)

Pour rendre nos métamodèles exploitables par un utilisateur final, nous avons intégré un atelier de modélisation graphique via l'outil **Sirius**. Cette approche par « points de vue » permet de s'affranchir de la syntaxe XMI, souvent ardue, au profit d'une manipulation intuitive des composants.

3.1 Visualisation du Catalogue

Nous avons mis en place un éditeur de diagramme pour le Catalogue permettant de définir graphiquement la bibliothèque de composants disponibles.

- **Fonctionnalités** : Création de types de composants, ajout de ports et spécification des caractéristiques techniques.
- **Choix ergonomiques** : Utilisation d'icônes distinctes pour différencier visuellement les entrées des sorties, facilitant la lecture rapide du catalogue.

■



FIGURE 4 – Éditeur graphique du Catalogue sous Sirius

3.2 Éditeur de Netlist

Le point de vue Netlist constitue le cœur logique de notre outil. Nous avons pu intégrer une représentation sous forme de graphe où les instances de composants sont reliées entre elles.

- **Fonctionnalités** : Instanciation des composants depuis le catalogue et création de connexions (nets) entre les ports.
- **Regard critique** : Bien que l'outil permette une création rapide, la gestion des croisements de fils dans les gros schémas reste un défi ergonomique propre à l'outil Sirius.

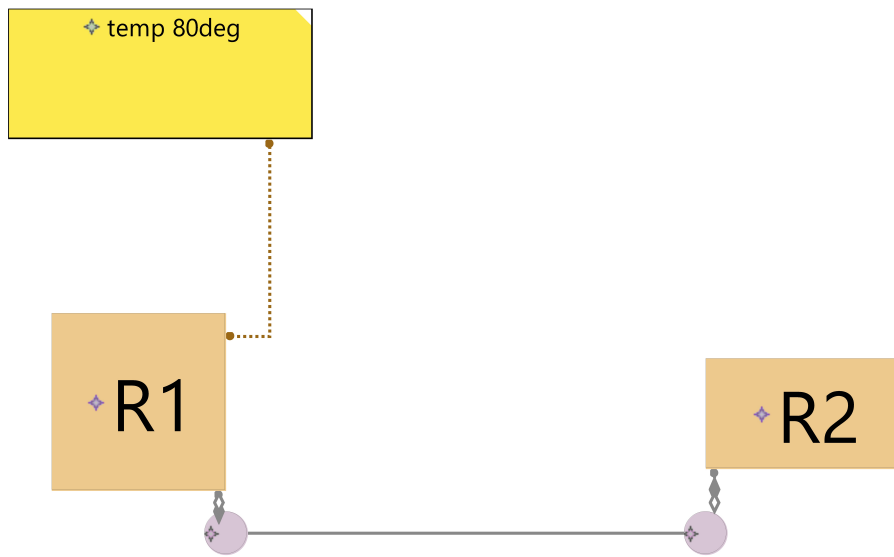


FIGURE 5 – Représentation graphique d'une Netlist (connectivité)

3.3 Visualisation du Layout

Enfin, nous avons développé un éditeur de placement physique (Layout). Contrairement à la Netlist, ce diagramme se concentre sur l'aspect spatial de la carte électronique.

- **Affichage graphique :** Nous avons pu intégrer sous Sirius une visualisation de la board, des différents composants ainsi que des couches layers qui constituent le circuit.

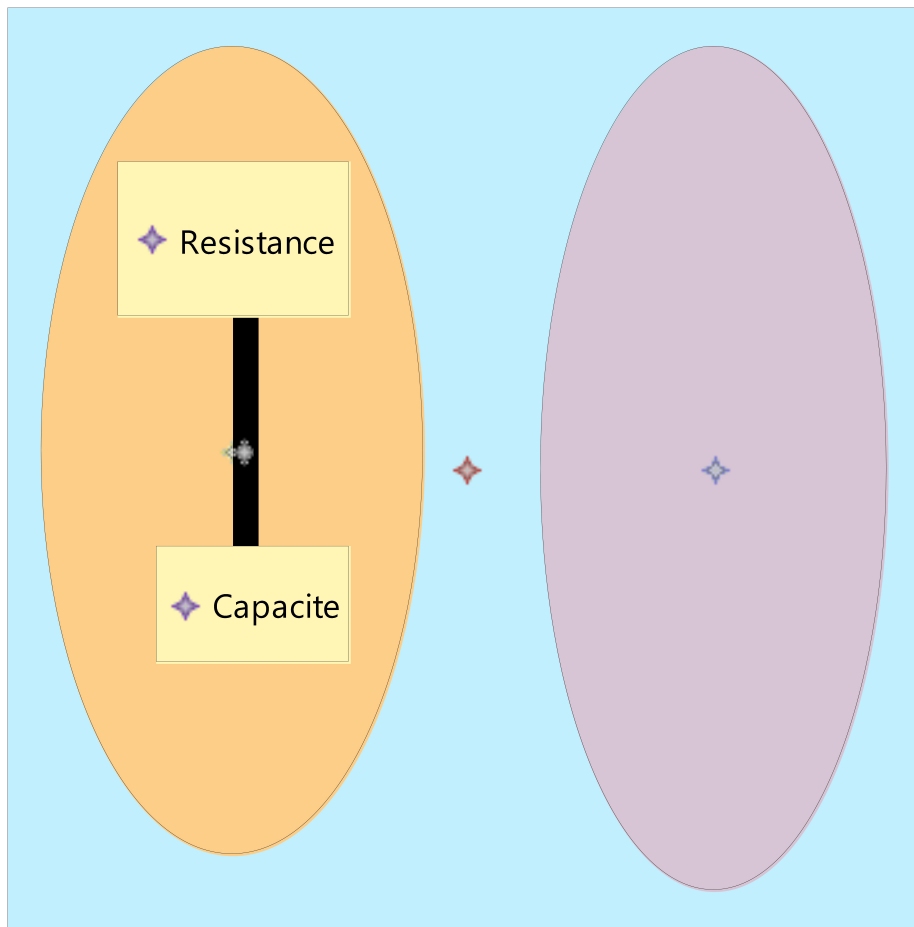


FIGURE 6 – Vue de placement spatial (Layout)

4 Chaîne de Transformations

L'un des piliers de notre projet est l'automatisation de la chaîne de production. L'Ingénierie Dirigée par les Modèles nous a permis de passer d'une description logique abstraite à des livrables concrets (physiques et textuels) sans intervention manuelle répétitive, garantissant ainsi une meilleure fiabilité des données.

4.1 Transformation Modèle-vers-Modèle (ATL)

La première étape de notre chaîne consiste en une transformation M2M utilisant le langage **ATL**. Nous avons automatisé le passage du modèle **Netlist** vers le modèle **Layout**.

- **Initialisation du placement** : L'objectif principal de cette transformation est de générer une esquisse de la carte électronique. Pour chaque instance de composant déclarée dans la Netlist, la transformation instancie un composant physique équivalent dans le Layout.
- **Logique de transformation** : Le script ATL parcourt les éléments de la Netlist et récupère les caractéristiques essentielles (ports, types) pour préparer la structure du Board.

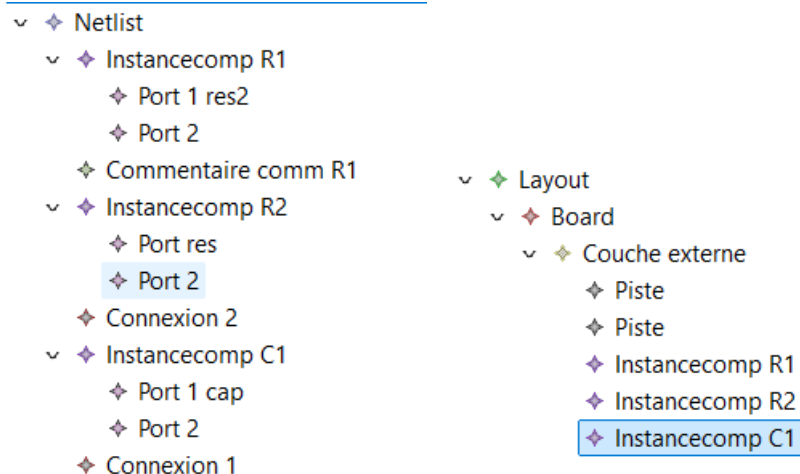


FIGURE 7 – Modèle Netlist d’origine et modèle Layout résultant

4.2 Transformations Modèle-vers-Texte (Acceleo)

Une fois les modèles consolidés, nous avons mis en œuvre des transformations M2T avec **Acceleo** pour extraire des informations exploitables.

4.2.1 Génération de la nomenclature (BOM)

À partir de la **Netlist**, nous avons conçu un générateur de fichier *Bill of Materials* (BOM).

- **Contenu** : Le template Acceleo itère sur l’ensemble des instances pour produire un rapport textuel listant les composants, leurs références au catalogue et le détail de leurs connexions.
- **Utilité** : Ce document constitue le lien direct avec la phase d’approvisionnement et de fabrication réelle du circuit.

4.2.2 Génération du rendu vectoriel (SVG)

Pour le modèle **Layout**, nous avons développé une génération vers le format **SVG** (*Scalable Vector Graphics*).

- **Principe** : Le générateur traduit les coordonnées (X, Y) et les dimensions des composants du métamodèle Layout en primitives graphiques XML.
- **Résultat** : Nous obtenons un plan technique du circuit, visualisable dans n’importe quel navigateur web, permettant de valider visuellement l’encombrement des composants sur la carte avant toute production physique.

Bill of Materials

ID	Type	Value	Contraintes	Remarques
R1	Resistance	ENSEEIH1HQ	d(self.Capacite) > 10mm redundancy(self) >= 2 && distanceToBoard(self) > 5 mm	temp max 80deg
R2	Resistance	ENSEEIH1HQ	d(self.Capacite) > 10mm redundancy(self) >= 2 && distanceToBoard(self) > 5 mm	
C1	Capacite	Google		

FIGURE 8 – Livrables générés : Fichier BOM

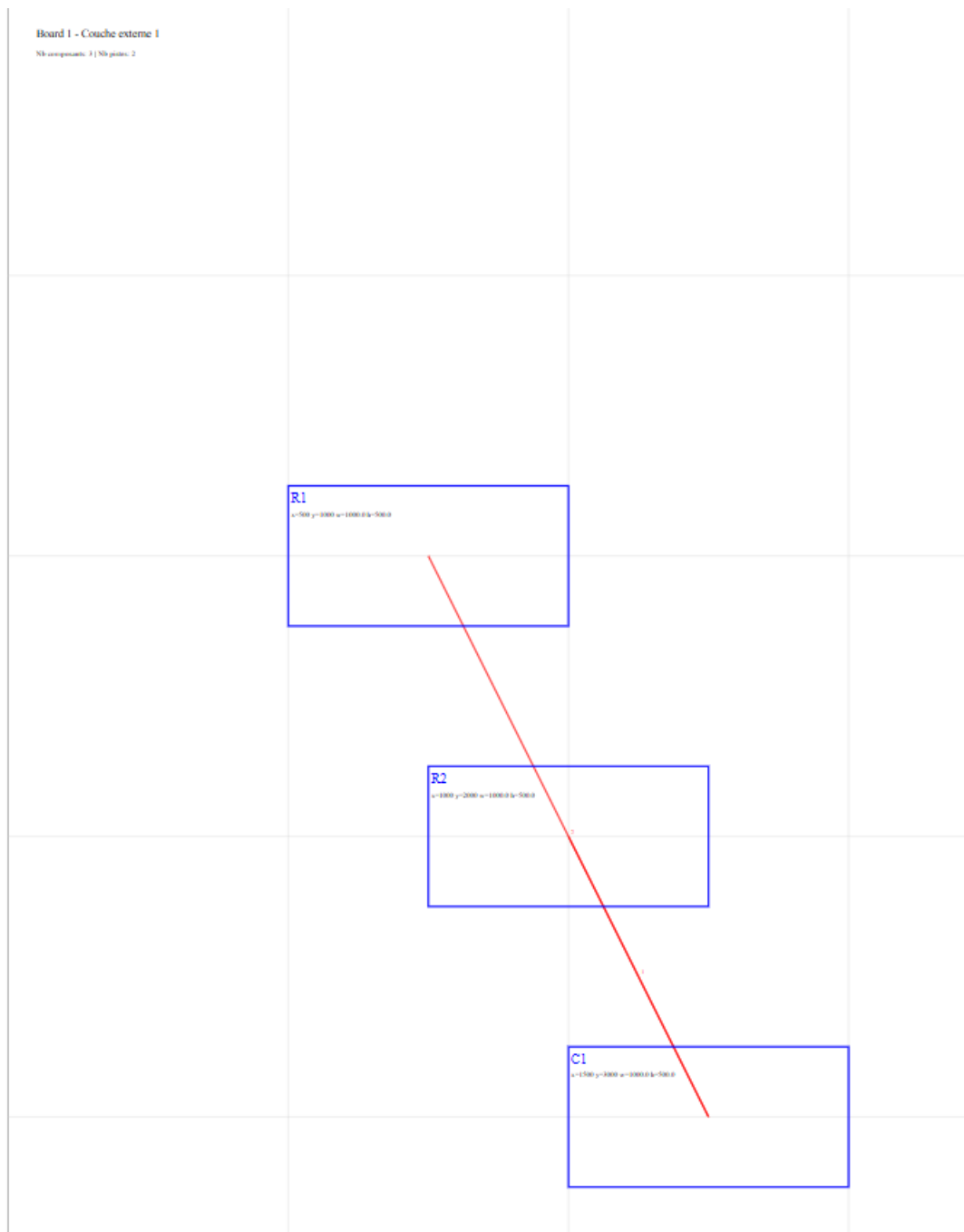


FIGURE 9 – Livrables générés : rendu SVG du circuit

5 Validation et Cohérence

La fiabilité d'un circuit électronique repose sur le respect de règles métier strictes. Nous avons mis en place un système de validation à deux niveaux : une définition textuelle simplifiée pour les experts métier et un moteur de vérification robuste pour la cohérence entre les modèles.

5.1 DSL de Contraintes (Xtext)

Pour faciliter la saisie des caractéristiques techniques des composants, nous avons développé un avec **Xtext**.

- **Objectif** : Cet éditeur textuel permet aux ingénieurs de définir les contraintes spécifiques à chaque composante du catalogue (par exemple, les limites de tension ou de température) sans manipuler de fichiers XML complexes.
- **Apport technique** : L'outil offre des fonctionnalités avancées telles que l'auto-complétion, la coloration syntaxique et la vérification d'erreurs en temps réel. Nous avons pu intégrer un parseur qui traite ces fichiers pour alimenter les métadonnées de notre catalogue.

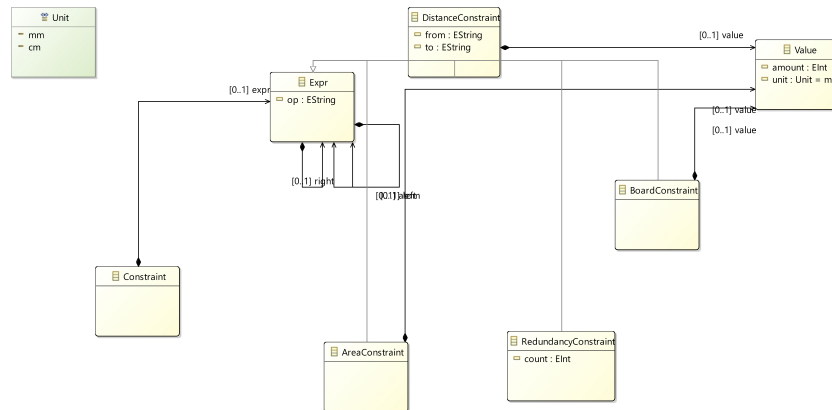


FIGURE 10 – Éditeur Xtext et grammaire des contraintes du catalogue

Pour permettre la définition textuelle des contraintes, nous avons conçu une grammaire personnalisée capable de manipuler des expressions logiques et des mesures physiques.

- **Structure logique** : La grammaire supporte les opérateurs booléens classiques (`,` `|`, `!`). Nous avons implémenté une hiérarchie de priorités (Or, And, Not) pour garantir que les expressions soient parsées sans ambiguïté, permettant de combiner plusieurs contraintes complexes pour un même composant.
- **Contraintes métier** : Nous avons intégré des primitives spécifiques au domaine de l'électronique :
 - **DistanceConstraint** : Pour gérer l'espacement entre deux composants.
 - **AreaConstraint** : Pour définir des zones de garde ou des surfaces vides.
 - **RedundancyConstraint** : Pour imposer un nombre minimal d'instances.
 - **BoardConstraint** : Pour les marges de sécurité par rapport au bord de la carte.
- **Flexibilité** : La grammaire gère les unités (mm, cm) et les opérateurs de comparaison, offrant une syntaxe proche du langage naturel (ex : `d(self, R1) > 5 mm`).

5.2 Vérifications Java : Alternative à l'OCL

5.2.1 Contraintes Métier des Modèles

Nous avons écrit des classes Java dédiées pour vérifier les contraintes des trois métamodèles.

- **Exemples de règles** : Vérification de redondance des instances de composant dans la Netlist, ou que deux composants ne se chevauchent pas physiquement dans le Layout.

- **Avantage** : L'implémentation Java nous a permis une gestion plus fine des contraintes, mieux que ce qui est offert par les ecores.

5.2.2 Vérificateur de Cohérence XMI (Layout vs Netlist)

L'un des points critiques de notre projet est la garantie que le fichier physique (Layout) correspond exactement à la spécification logique (Netlist).

- **Fonctionnalité** : Nous avons développé un utilitaire Java capable de charger un fichier .xmi de Layout et de le comparer à sa Netlist source.
- **Vérification** : Le programme s'assure que chaque connexion (net) entre deux composants est physiquement réalisable par des pistes dans le layout, garantissant ainsi l'intégrité du design final.

6 Regard critique

Cette section dresse un bilan analytique des solutions techniques retenues et de leur viabilité dans un contexte de production industrielle.

6.1 Ergonomie et Passage à l'échelle

L'intégration de **Sirius** a considérablement amélioré l'ergonomie globale du système en offrant une abstraction graphique indispensable. Cependant, nous avons identifié des limites concernant le passage à l'échelle (*scalability*) :

- **Lisibilité** : Sur des circuits complexes comportant des centaines de composants, la représentation sous forme de diagrammes devient rapidement saturée. La gestion manuelle du routage des fils dans Sirius manque de fluidité par rapport à des outils de CAO spécialisés.
- **Performance** : Nous avons observé que la synchronisation en temps réel entre les modèles Ecore et les représentations graphiques peut induire des ralentissements lorsque la profondeur du métamodèle est importante.

6.2 Manquements et Améliorations possibles

Le principal manquement de notre réalisation réside dans l'absence d'intégration native d'OCL (*Object Constraint Language*) au sein d'Ecore.

- **Le choix du Java** : Bien que l'alternative Java nous ait offert une plus grande flexibilité pour le débogage et la personnalisation des messages d'erreur, elle sacrifie l'aspect déclaratif et la portabilité des contraintes.
- **Amélioration** : Une évolution pertinente consisterait à encapsuler nos validations Java dans des *Checkers* Eclipse afin de notifier l'utilisateur directement via l'interface « Problems » en temps réel, renforçant ainsi l'aspect « outil industriel ».

7 Conclusion et Bilan

Ce projet nous a permis d'appréhender l'intégralité du cycle de vie d'un système piloté par les modèles, de la définition sémantique (Ecore) à la génération de livrables concrets (Acceleo/SVG).

7.1 Bilan technique et difficultés

La réalisation de ce projet a été jalonnée de défis techniques qui ont nécessité une adaptation constante de notre part.

- **Conception des métamodèles** : La phase initiale de modélisation sous Ecore a été l’une des étapes les plus complexes. Il a fallu anticiper l’ensemble des besoins pour les trois vues (Catalogue, Netlist, Layout) tout en assurant une structure permettant la traçabilité. Un choix de conception erroné au départ se répercutant sur toutes les transformations ultérieures, cette étape a demandé de nombreuses itérations pour obtenir une base solide.
- **Limitations de Sirius** : Bien que puissant pour la visualisation, l’outil Sirius a présenté des limites ergonomiques majeures. La gestion des bordures de la « board » et le positionnement automatique des ports sur les composants ont été difficiles à stabiliser.
- **Instabilités d’Eclipse** : Un frein majeur à notre productivité a été l’instabilité chronique de l’environnement de développement Eclipse.
- **Intégration finale de Xtext** : Concernant le DSL de contraintes, si la grammaire et le parseur ont été développés avec succès, leur intégration complète dans la chaîne de production finale n’a pas pu être finalisée. La liaison entre les contraintes éditées textuellement et leur application automatique sur les modèles existants représentait un verrou technique que le temps imparti ne nous a pas permis de lever totalement. Nous avons donc privilégié la robustesse des validations Java pour garantir la livraison d’un projet fonctionnel.

8 Description détaillée du rendu

Le livrable est organisé en une structure de dossiers clairs regroupant les différentes couches de l’ingénierie dirigée par les modèles mise en œuvre. Voici le détail du contenu :

8.1 Dossier : metamodels

Ce dossier contient la définition structurelle de notre système. Chaque métamodèle est fourni avec son fichier source Ecore et son modèle de génération.

- `catalogue.ecore` : Définition des composants et de leurs caractéristiques.
- `netlist.ecore` : Structure de connectivité logique.
- `layout.ecore` : Spécification du placement physique et des couches.

8.2 Dossier : sirius

Ce dossier centralise les aspects de visualisation graphique pour l’ensemble des métamodèles.

- **Dossiers .odesign** : Contiennent la configuration des points de vue (représentations graphiques, outils de création, icônes) pour le Catalogue, la Netlist et le Layout.
- **Dossiers .sample** : Modèles d’exemples (`.xmi`) permettant de tester immédiatement les éditeurs graphiques avec des données pré-remplies.

8.3 Dossier : xtext

Ce dossier est dédié à la syntaxe concrète textuelle pour la gestion des contraintes.

- `dsl.xtext` : Ce fichier contient la grammaire complète permettant de définir textuellement les contraintes de distance, de zone et de redondance pour les composants du catalogue.

8.4 Dossier : validation

Ce dossier regroupe toute l’intelligence de vérification développée en Java (alternative à l’OCL). Il est organisé par paquets de validation pour chaque métamodèle :

8.5 Dossiers : acceleo et atl

- **acceleo** : Contient les modules .mtl pour la génération des fichiers du BOM et SVG.
- **atl** : Contient le fichier de transformation .atl pour la génération automatique du modèle de Layout à partir de la Netlist.

8.6 Dossier : vérification de cohérence

Ce dossier est spécifique à la validation croisée entre les modèles.

- **Logiciel de cohérence** : Contient le code Java permettant de charger et comparer un fichier XMI de Layout avec son fichier Netlist source. Il garantit que chaque composant placé et chaque piste tracée physiquement possèdent une existence logique correspondante.